

Name : YASHVI MEHTA

Roll no. : 231071079

LY BTech Computer

Batch D

Date: 2026-08-06

## Cross Platform App Dev Lab 3

**AIM** - To study and implement adaptive and responsive UI design techniques in Flutter, and build a Flutter application whose layout automatically adjusts to different screen sizes, orientations, and device types (mobile, tablet, web/desktop).

### THEORY -

#### Adaptive Design vs Responsive Design

##### Responsive Design -

Responsive design is the technique of creating a UI that **automatically changes its size, position, spacing, and arrangement** according to the available screen space. It mainly focuses on making the interface fit different screen sizes.

##### Adaptive Design -

Adaptive design is the technique of creating a UI that **selects or changes the layout based on the device, screen characteristics, orientation, or available space**. It may provide significantly different layouts for mobile, tablet, and desktop/web devices.

#### Difference between Responsive and Adaptive Design

| Feature    | Responsive Design                          | Adaptive Design                                               |
|------------|--------------------------------------------|---------------------------------------------------------------|
| Meaning    | UI adjusts continuously to available space | UI switches between layouts designed for different conditions |
| Main focus | Screen size and available space            | Device type, screen size, orientation, input method, etc.     |

|                    |                                                      |                                                                   |
|--------------------|------------------------------------------------------|-------------------------------------------------------------------|
| Layout             | Usually fluid                                        | Usually uses predefined layouts                                   |
| Example            | A column becomes narrower as the screen gets smaller | Mobile uses bottom navigation while tablet uses a navigation rail |
| Flutter techniques | MediaQuery, LayoutBuilder, Flexible, Expanded, Wrap  | Breakpoints, conditional layouts, device/orientation checks       |
| Best suited for    | Different screen sizes                               | Different device categories and experiences                       |

In practice, Flutter applications often use both approaches together to create interfaces that work well across mobile, tablet, web, and desktop.

## Key Widgets and Classes -

### 1. MediaQuery

MediaQuery provides information about the **current screen and device environment**.

It can be used to obtain:

- Screen width and height
- Device pixel ratio
- Orientation
- Text scaling information
- System padding and safe areas
- Other display characteristics

Example -

```
double screenWidth = MediaQuery.of(context).size.width;
```

```
double screenHeight = MediaQuery.of(context).size.height;
```

```
Orientation orientation = MediaQuery.of(context).orientation;
```

## 2. LayoutBuilder

LayoutBuilder provides the **constraints available to a particular widget**.

Unlike MediaQuery, which gives information about the overall screen, LayoutBuilder tells us how much space is available to the widget in its parent layout.

### Example

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth < 600) {  
      return const MobileLayout();  
    } else {  
      return const DesktopLayout();  
    }  
  },  
)
```

## 3. OrientationBuilder

OrientationBuilder allows the UI to respond to changes between **portrait and landscape orientation**.

### Example

```
OrientationBuilder(  
  builder: (context, orientation) {  
    if (orientation == Orientation.portrait) {  
      return const Text("Portrait Layout");  
    } else {
```

```

    return const Text("Landscape Layout");
  }
},
)

```

The application can change the number of columns, arrangement of controls, or spacing depending on the orientation.

#### 4. Expanded, Flexible and Wrap

Expanded and Flexible allow children of a Row or Column (built in Experiment 2) to grow or shrink proportionally using the flex property, instead of using fixed pixel widths/heights. Wrap automatically moves children to the next line when there is not enough horizontal space, which is useful for responsive grids of chips, tags, or cards.

Example -

```

Row(
  children: [
    Expanded(flex: 2, child: Container(color: Colors.blue)),
    Expanded(flex: 1, child: Container(color: Colors.red)),
  ],
)

```

#### Breakpoints for Screen Layouts

**Breakpoints** are predefined screen-width values at which an application changes its layout or navigation structure.

They allow developers to categorize available screen space into layouts such as mobile, tablet, and desktop.

A commonly used approach is:

| Screen Width | Device Category | Typical Layout                           |
|--------------|-----------------|------------------------------------------|
| < 600 px     | Mobile          | Single-column layout, compact navigation |
| 600–1024 px  | Tablet          | Two-column or expanded layout            |
| > 1024 px    | Desktop/Web     | Multi-column layout, side navigation     |

---

## APP OVERVIEW -

### App title - Campus Companion

A student-focused campus information dashboard built as a Flutter MVP for LY B.Tech Computer Engineering (VJTI). It demonstrates clean UI design and meaningful usage of Flutter's core layout widgets: Row, Column, and Stack.

#### Key Features -

**1. Home Dashboard:**

Displays a personalized greeting, date, quick snapshot of today's classes, and a 6-feature grid (3 active + 3 "Coming Soon" badges).

**2. Weekly Timetable:**

Features a day selector (MON–FRI) populated with the actual AY 2026–27 LY CE timetable, displaying class timings, subjects, faculty, and room/lab numbers.

**3. Professor Directory:**

Lists CE department faculty with a detail view for room locations, office hours, email, and subjects taught.

#### 4. Campus Events:

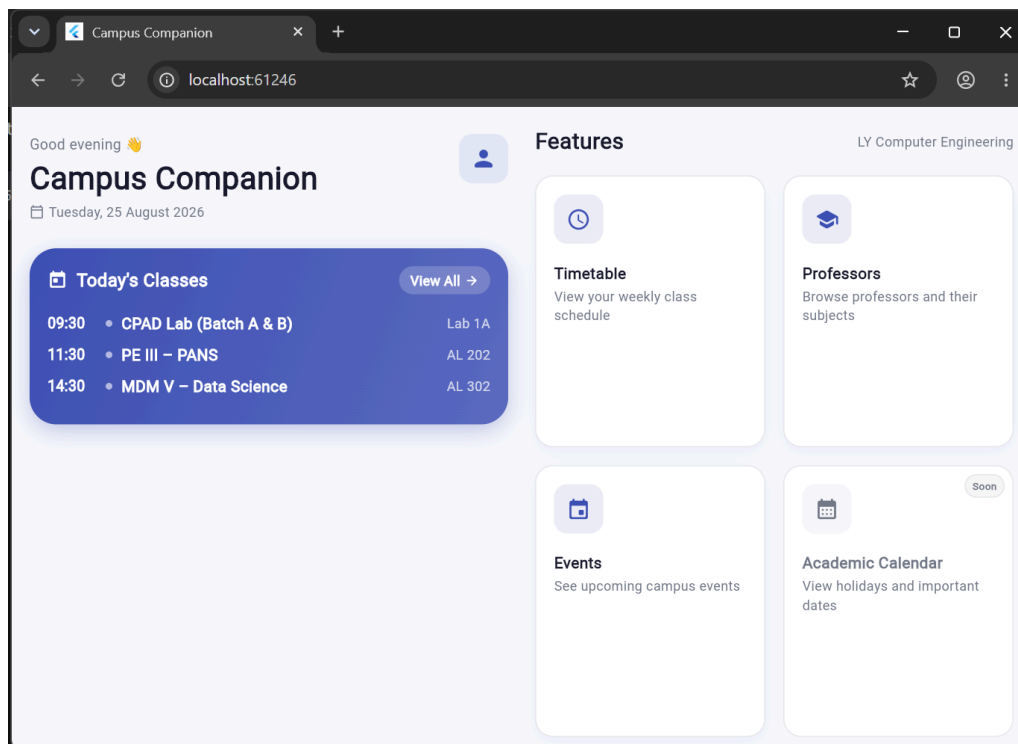
Displays upcoming campus events (Tech, Cultural, Hackathons) with detailed views and category overlays.

### REPO LINK FOR CODE -

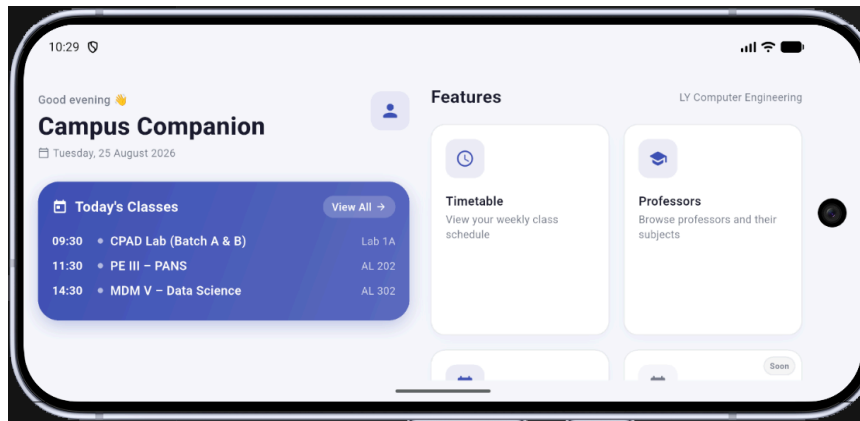
[https://github.com/YashviMehta03/CPAD\\_lab\\_repos/tree/main/lab2\\_app](https://github.com/YashviMehta03/CPAD_lab_repos/tree/main/lab2_app)

### SCREENSHOTS -

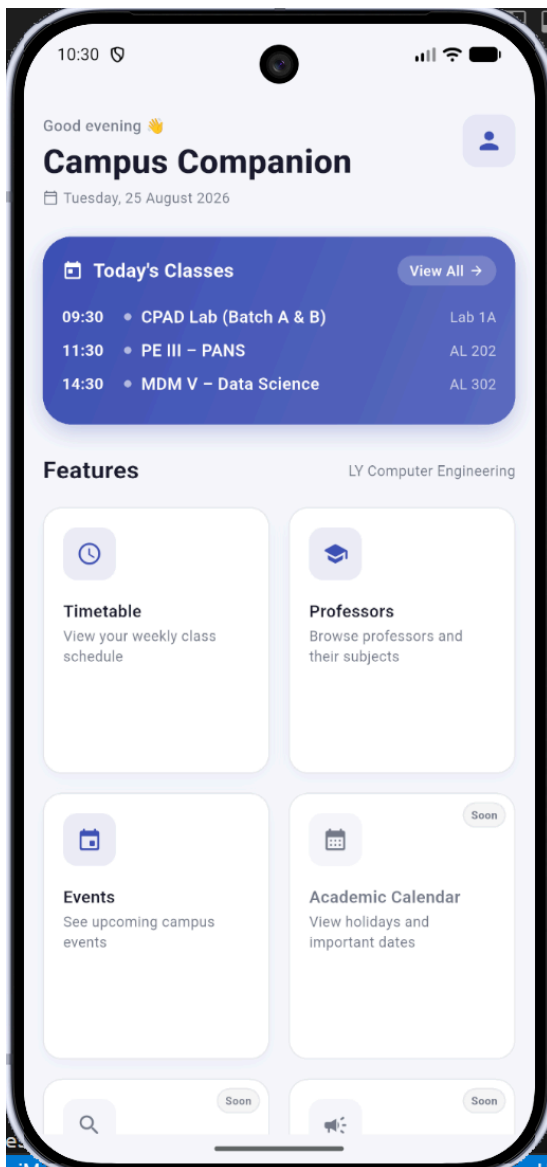
#### 1) Tablet Screen (>=600 dp)



## 2) Mobile landscape layout -



## 3) Mobile portrait layout -



## Conclusion -

Adaptive and responsive UI design enables a Flutter application to provide a consistent and usable experience across **mobile phones, tablets, web browsers, and desktop devices**.

**MediaQuery** provides screen information, while **LayoutBuilder** responds to available constraints. **OrientationBuilder** handles portrait and landscape changes, and **Expanded**, **Flexible**, and **Wrap** help widgets use available space efficiently. **Breakpoints** can then be used to switch between mobile, tablet, and desktop layouts. Together, these techniques make Flutter applications flexible, scalable, and user-friendly across different devices.